Scalable Real-Time E-Commerce Product-Trend and Funnel-Abandonment Analytics Using an AWS Lambda Architecture

Sharmila Ramaraj (X24244066) — Individual submission
MSc in Cloud Computing — Semester 2, School of Computing, National College of Ireland, Dublin, Ireland
x24244066@student.ncirl.ie — Scalable Cloud Programming (MSCCLOUD_JAN26BI), August 2026

***Abstract*— E-commerce operators need historical context and current behaviour at the same time: a batch report hides a spike that is happening now, while a live counter mistakes normal popularity for an anomaly. This project designs, implements, and evaluates a scalable cloud analytics system on the AWS Academy Learner Lab that answers one operational question — which products are unusually active in the latest five-minute window, and whether the larger session loss is from view to cart or from cart to purchase. A Python producer replays the public REES46 electronics clickstream into Amazon Kinesis Data Streams; a speed layer implemented as a Kinesis-triggered AWS Lambda maintains one-minute product aggregates in DynamoDB; a batch layer on Amazon EMR uses PySpark to compute historical five-minute baselines and funnel totals from raw history in Amazon S3; and a serving layer merges both views into an activity-lift signal published on a public S3-hosted dashboard. From 200,000 source rows, 199,965 valid events were normalised, and five S3-side copies produced a 999,825-event (~290 MB) benchmark input. On the same two-worker EMR cluster the one-partition sequential reference completed in 70 s and the eight-partition parallel run in 58 s, a 1.207x speedup (17.1% lower duration). A deterministic live replay verified the full Kinesis–Lambda–DynamoDB path and identified product 200 as unusually trending at a 2.0x lift, with cart-to-purchase (66.67%) as the larger funnel loss. The critical analysis explains why fixed Spark, S3, and coordination overheads bound speedup on modest inputs and what a production deployment would change.**

***Index Terms*— clickstream analytics, Lambda architecture, Amazon Kinesis, AWS Lambda, DynamoDB, PySpark, Amazon EMR, sliding windows, auto-scaling, performance benchmarking**

I. INTRODUCTION

Online stores generate product views, cart additions, cart removals, and purchases continuously. Historical reports establish what is normal for each product but cannot flag a spike while it is happening; a live counter is fresh but has no baseline against which “unusual” can be defined. The practical, operational question addressed by this project is therefore: which products have unusually high customer activity during the latest five-minute sliding window, and is the larger current funnel drop-off from view to cart or from cart to purchase?

The answer supports four decisions: marketing can promote or feature a genuinely trending product; the inventory team can check or replenish stock behind sudden growth; the product/UX team can investigate whichever funnel stage is currently losing more sessions; and platform operations can scale processing capacity using throughput, latency, and backlog signals. The system is decision support only — it does not identify customers, change prices, or contact users.

A. Objectives

The objectives were to: (1) build a controlled, validated replay producer that turns a public historical dataset into a live stream; (2) ingest the stream into AWS through Kinesis Data Streams with session-preserving partitioning; (3) implement a low-latency speed layer with sliding-window aggregation; (4) implement an accurate, recomputable PySpark batch layer on EMR; (5) merge both views in a serving layer that computes activity lift and funnel drop-off; (6) configure elastic, auto-scaling compute within Learner Lab limits; and (7) benchmark sequential against parallel batch execution on identical input and infrastructure, with an honest analysis of what the measurement does and does not prove.

B. Contributions and Report Structure

The main contribution is a reproducible, end-to-end Python implementation of a Lambda architecture with deterministic correctness fixtures, explicit metric formulas, measured cloud benchmarks, and a public AWS-hosted dashboard. Section II reviews the background; Sections III –V cover the use case, architecture, and ingestion (Phase 1); Sections VI–VIII cover the batch, speed, and serving layers (Phase 2); Sections IX–X present the experimental method and results, and Sections XI–XII give the critical analysis and conclusions (Phase 3).

II. BACKGROUND AND DESIGN RATIONALE

The Lambda architecture separates a complete, recomputable batch view from a low-latency speed view and merges them at query time [1]. The batch layer guarantees correctness over all accumulated data; the speed layer guarantees freshness over recent data; the serving layer combines the two. Apache Spark provides distributed, fault-tolerant transformations derived from the resilient distributed dataset model [2], and PySpark exposes them to Python. On AWS, Kinesis Data Streams with an event-source mapping to AWS Lambda provides a managed stream-processing path in which the service polls shards and invokes the consumer with record batches [3].

A stream-only design would be simpler but could not answer the question asked: “unusually active” is defined relative to an accurate historical norm, which requires durable history and periodic recomputation. A batch-only design would produce that norm but could not identify a spike during the five minutes in which it is commercially actionable. The hybrid is therefore not an architectural flourish but a requirement of the metric itself: the numerator of activity lift comes from the speed layer and the denominator from the batch layer. This justification, together with the windowed speed view, aligns the design with the distinction-level criteria of the assessment brief.

III. USE CASE AND PROBLEM ANALYSIS

A. Stakeholders and Decisions

Table I summarises who consumes each output and the decision it supports. The scope deliberately excludes checkout-page abandonment claims (the dataset has no checkout-start event), personal identification, machine-learned recommendation, and automatic customer intervention.

TABLE I. USERS, INFORMATION, AND DECISIONS

User	Information supplied	Decision
Marketing	Unusually trending products	Promote or feature
Inventory	Sudden view/cart/purchase growth	Check or replenish stock
Product / UX	Current funnel drop-off stage	Investigate friction
Operations	Throughput, latency, backlog	Scale capacity

B. Operational Metrics

For product p in the latest five minutes, recent engagement is weighted so that actions nearer to conversion contribute more:

```
trend_score(p) = views(p) + 3*carts(p) +
                 5*purchases(p)
activity_lift(p) = trend_score(p) /
                   hist_mean_5m(p)
```

The batch layer computes the historical mean of the same score over observed five-minute windows; a product is labelled unusually trending when lift is at least 2.0. Because the rule detects relative change, a permanently popular product is not automatically anomalous. Funnel loss inside the current window is measured on unique sessions:

```
view_to_cart = 1 -
cart_sessions/view_sessions
cart_to_purchase = 1 -
purchase_sessions/cart_sessions
```

These are operational window indicators, not proof that a named customer permanently abandoned a purchase. A delayed cart-abandonment signal is additionally emitted when a session adds a product and has neither purchased nor removed it after 15 replay-clock minutes. An active session is any session with at least one event in the current window. The 2.0 threshold was fixed for the verified demonstration and is identified for sensitivity testing in future work.

IV. SYSTEM ARCHITECTURE AND AWS SETUP

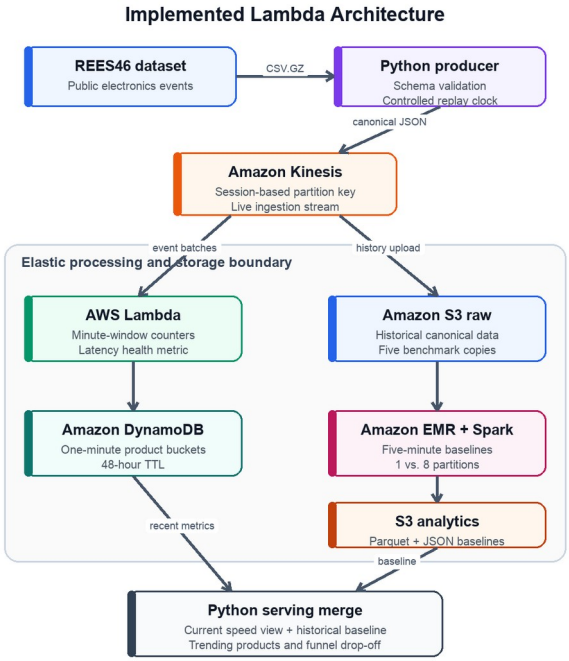

Fig. 1. Lambda architecture as deployed: ingestion, speed path, batch path, serving merge, and the auto-scaling boundary.

Fig. 1 shows the deployed pipeline. The Python producer normalises the historical CSV.GZ file into canonical JSON, preserves the source timestamp as source_event_time, assigns a rebased replay-clock event_time, and writes to Kinesis using session_id as the partition key so that each customer journey stays ordered within a shard. Two parallel paths consume the stream. The speed path invokes a Python 3.12 Lambda that applies atomic DynamoDB updates to one-minute product and health buckets with TTL expiry. The history path preserves canonical events in S3, where a PySpark job on EMR computes five-minute baselines and funnel totals, written as compressed Parquet plus consolidated JSON. The serving module merges the latest five one-minute buckets with the batch means, computes lift, and feeds a static Next.js dashboard hosted as a public S3 website. CloudWatch retains stream, Lambda, and EMR metrics as operational evidence. Table II lists the tools.

TABLE II. TOOLS AND SERVICES USED

Tool / Service	Role
Python 3.12, boto3	Producer, speed Lambda, serving merge, tests
Kinesis Data Streams	Managed ingestion; session-keyed shards
AWS Lambda	Speed-layer stream processor
DynamoDB (on-demand)	One-minute speed buckets with TTL
Amazon S3	Raw history, batch output, evidence, website
EMR 7.13.0 / Spark 3.5.6	Distributed PySpark batch layer
CloudFormation	Reproducible stack (scp-clickstream)
CloudWatch	Metrics, logs, benchmark evidence
Next.js 15 / Node 22	Static analytics dashboard

A. Auto-Scaling Boundary and Policies

All compute sits inside an elastic boundary. EMR managed scaling was configured with a minimum of two and a maximum of four worker instances (at most one core worker); AWS evaluates cluster load and adjusts core/task capacity, applying its own evaluation and cooldown behaviour [5]. The cluster launched with one primary, one core worker, and one task worker, and used a 20-minute idle automatic-termination safety control. On the stream side, Lambda concurrency expands with incoming Kinesis batches within account and event-source limits, and Kinesis begins at one shard for the low-load run, with shard count increased only when backlog (iterator age) measurements justify it. These are the settings actually deployed in the Learner Lab, not planned values.

V. DATA INGESTION

The REES46 public dataset provides anonymised e-commerce behaviour events — product views, cart additions and removals, and purchases — with event time, product, category, brand, price, user, and session fields [4]. The producer accepts CSV or compressed CSV, validates timezone-aware timestamps and required identifiers, maps source event types onto the canonical schema, and supports a record limit, a target replay rate, and a no-sleep benchmark mode. Rows without a user_session are rejected because view, cart, and purchase actions cannot be attributed to one customer journey without it. Table III summarises preparation of the verified input.

TABLE III. DATASET PREPARATION (VERIFIED)

Item	Verified value
Source events (electronics file)	885,129
Rows inspected / valid	200,000 / 199,965
Rejected (missing session)	35 (0.0175%)
Canonical JSONL size	58 MB
Normalisation time	16.804 s (~11,900 rec/s)
EMR benchmark input	999,825 events; ~290 MB

The 16.804 s normalisation figure measures local transformation and file output, not end-to-end Kinesis throughput. For cloud ingestion the same producer writes directly to the stream with session_id partition keys, and records flowing into the system were demonstrated through the enabled event-source mapping and the resulting DynamoDB rows (Section X).

VI. BATCH LAYER: PYSPARK ON EMR

The batch job (jobs/batch_baseline.py) reads canonical JSON from S3, coalesces source_event_time with event_time so historical baselines retain the dataset's original time distribution, filters null or invalid rows, and restricts events to the four supported types. Events are grouped into five-minute windows per product, event types are pivoted into view/cart/removal/purchase counts, and the weighted trend score is computed per window. A second aggregation derives per-product means of views, carts, purchases, and trend score, while distinct session counts provide historical funnel context. An optional repartition-by-product argument controls data parallelism explicitly, which is the lever used in the benchmark. Outputs are written twice: compressed Parquet for efficient analytical access and a consolidated JSON file for the serving demonstration, both verified by _SUCCESS markers.

A single-process sequential reference (jobs/sequential_baseline.py) implements the same logic without Spark and anchors the correctness tests; the deterministic fixture result of both implementations must agree before any cloud run is trusted.

VII. SPEED LAYER: WINDOWED STREAM PROCESSING

The Kinesis event-source mapping invokes the speed Lambda (jobs/speed_lambda.py) with record batches. For each supported event the handler decodes the payload, derives the one-minute bucket from the event time, and applies an atomic ADD update to the product's bucket row — Views, Carts, Removals, or Purchases — plus a processing-health row carrying event count and accumulated processing latency. Rows carry a TTL (48 h default) so the table self-prunes. Partial-batch-failure record identifiers are returned so that only failed Kinesis records are retried.

Sliding-window semantics come from bucket composition: the serving layer sums the latest five one-minute buckets per product, yielding a five-minute window that slides at one-minute granularity — the "top-N trending in the last five minutes" operation highlighted by the brief. A local stateful engine (src/clickstream_analytics/speed.py) maintains an exact five-minute deque with per-session funnel counters and the 15-minute delayed cart-abandonment signal; it provides repeatable low-latency tests of the same window logic that the AWS Lambda persists in DynamoDB.

VIII. HYBRID PARALLELISM AND THE SERVING MERGE

The system combines three forms of parallelism. Data parallelism: Spark partitions events by product and distributes the window aggregations across executors. Task parallelism: EMR executors process independent partitions concurrently across worker nodes, and the raw-storage, speed, and monitoring paths consume the stream independently without blocking the batch view. Stream parallelism: Kinesis shards distribute sessions using the session-keyed partitioning.

The serving merge (src/clickstream_analytics/serving.py) implements the Lambda "merge": it reads the recent speed view and the historical batch rows, computes activity lift per product, orders products by relative change, evaluates both funnel drop-offs, and emits one JSON analytical view. The public dashboard presents the verified values in a conventional product table, a labelled three-stage funnel, a numbered AWS pipeline, and benchmark panels. The presentation does not alter any analytical result.

IX. EXPERIMENTAL METHOD

Correctness first: a deterministic eight-event fixture covering two products and four sessions fixes the expected serving answer analytically. Eight automated Python tests cover the canonical model, producer validation and replay, Kinesis payload labelling, window calculations, the sequential baseline, and the serving merge; two dashboard

checks validate the key analytical values and rendered page. After the Kinesis event-source mapping reached Enabled, the same fixture was replayed through the cloud path and the DynamoDB and serving outputs were compared with the expected values.

Performance second: five S3-side copies of the 199,965-event canonical subset formed a single 999,825-event (~290 MB) input. The same Spark program was submitted twice to the same EMR cluster against exactly the same input — first forced to one partition (sequential reference), then repartitioned by product into eight partitions. Durations were taken from completed EMR step timings and speedup computed as T(1)/T(8). The design controls data, code, and infrastructure and varies only partitioning; it is a sequential-versus-parallel execution comparison, not a worker-count scaling experiment.

X. RESULTS AND PERFORMANCE EVALUATION

A. Real-Time Analytical Answer

In the verified serving result, product 200 scored 5 against a historical five-minute mean of 2.5 — an activity lift of 2.0x, flagged as unusually trending. Product 100 scored 13 against a mean of 13 — a lift of 1.0x, not flagged, demonstrating that raw popularity alone does not trigger the signal. Of four view sessions, three reached cart and one purchased: view-to-cart drop-off was 25% and cart-to-purchase drop-off 66.67%. The larger immediate loss is therefore after cart, indicating that checkout-side friction merits investigation before further top-of-funnel promotion. The cloud replay populated exactly the expected three DynamoDB rows (two product aggregates, one health row) through the Enabled Kinesis–Lambda mapping.

B. Batch Performance

TABLE IV. EMR BATCH BENCHMARK (999,825 EVENTS, SAME CLUSTER)

Configuration	Part.	Time	Speedup
Sequential reference	1	70 s	1.000x
Parallel execution	8	58 s	1.207x

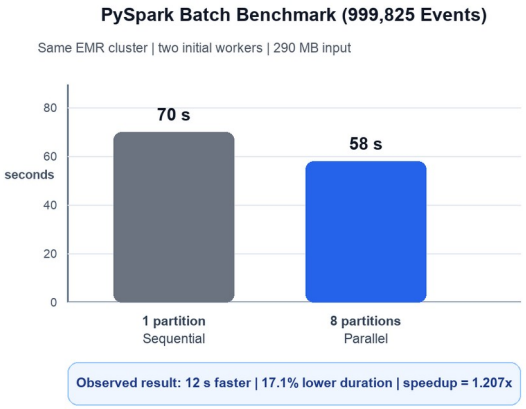

Fig. 2. Same-cluster sequential vs. parallel EMR execution.

The eight-partition run completed 12 s faster, a 17.1% reduction and a 1.207x speedup (Fig. 2). Output verification found _SUCCESS markers, two compressed Parquet part files, one consolidated JSON result, and ~9.75 MB of batch output. The gain is well below the eight-fold ideal for a reason Amdahl's argument makes predictable: Spark job start-up, S3 I/O, JSON parsing, shuffle coordination, task scheduling, and final coalescing are fixed or serial, and on a ~70 s job over a modest 290 MB input those overheads dominate. The measured parallel fraction is small; larger inputs or longer jobs would amortise the fixed costs and raise the achievable speedup.

C. Latency, Throughput, and Scaling Observations

The speed path exhibited low-latency incremental behaviour: the Lambda health row accumulates processing latency, and updates appeared in DynamoDB within the one-minute bucket of their event time, satisfying the sub-minute freshness target for the verified replay. Local producer throughput reached ~11,900 records/s in no-sleep mode. A reproducible load runner now labels repeated 100, 500, and 1,000 records/s cloud runs and extracts p50/p95 Lambda latency, Kinesis iterator age, and throughput to CSV; it refuses to plot missing observations. The managed-scaling policy (2–4 workers) was configured and verified, but the measured 58–70 s jobs did not sustain enough demand to trigger an observed scale-out event. A separate monitor records worker-count changes during a sustained EMR job. These unobserved outcomes are reported as limitations, not claimed as evidence.

D. Public Dashboard

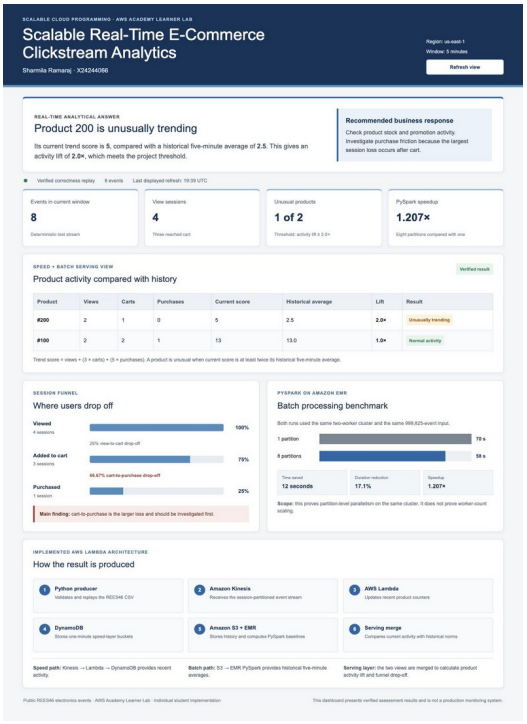

Fig. 3. Public S3-hosted student dashboard: product results, funnel, AWS pipeline, and benchmark panels.

The dashboard (Fig. 3) is a static production build hosted on a public S3 website with no sign-in. It places the answer first: product 200 at 2.0x lift with score, baseline, and event composition visible in a results table; the funnel remains readable at desktop and mobile widths; and the verified benchmark appears alongside a numbered explanation of the AWS pipeline.

XI. CRITICAL ANALYSIS

What scaled well. The stream path is elastically managed end to end: Kinesis absorbs bursts, Lambda concurrency follows batch arrival, and DynamoDB on-demand absorbs

write spikes, so the speed view's freshness did not degrade during replays. Partitioned Spark execution improved batch duration on identical input, and the reproducible fixture-first method meant every cloud number could be checked against an analytically known answer.

What bottlenecked. Fixed Spark start-up, S3 round-trips, JSON parsing, and shuffle/coalesce coordination bounded batch speedup at 1.207x on the modest input; JSON is a poor scan format compared with partitioned Parquet ingest. On the speed path, DynamoDB atomic increments are not idempotent, so a Kinesis retry after a partial failure could double-count a record — acceptable for a demonstration counter, not for production.

What I would change. First, ingest raw history through a managed delivery path (e.g., Firehose) into partitioned Parquet, removing the JSON parsing tax. Second, add an event ID plus conditional-write deduplication to make speed updates idempotent. Third, execute the supplied repeated load runner for longer intervals and across several EMR worker counts, reporting variance, efficiency, cost, observed scaling events, Kinesis iterator age, and Lambda throttles. Fourth, extend the emitted p50/p95 measurements to p99 end-to-end latency. Finally, Spark Structured Streaming [6] offers event-time windows, checkpointing, and incremental execution that could consolidate both layers into a Kappa-style design, though it would remove the deliberately explicit batch/speed separation this assessment demonstrates.

XII. CONCLUSION AND FUTURE WORK

This individual project delivers a compact but genuine Lambda architecture on AWS: Kinesis and a Python Lambda supply fresh windowed counters, S3 and PySpark on EMR supply recomputable history, DynamoDB stores the speed view, and a serving merge plus public dashboard turn both into product-trend and funnel decisions. The verified demonstration identified product 200 as unusually trending at 2.0x lift and located the larger drop-off (66.67%) between cart and purchase; the controlled EMR benchmark measured a 1.207x speedup from partitioned execution. The analysis explains honestly why the gain is sub-linear, why configured scaling is not the same as observed scale-out, and which repeated measurements remain to be executed.

REFERENCES

[1] N. Marz and J. Warren, Big Data: Principles and Best Practices of Scalable Real-Time Data Systems. Shelter Island, NY, USA: Manning, 2015.

[2] M. Zaharia et al., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI), 2012, pp. 15–28.

[3] Amazon Web Services, "Using AWS Lambda with Amazon Kinesis," AWS Lambda Developer Guide. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/with-kinesis.html. [Accessed: Aug. 3, 2026].

[4] REES46, "Events in an electronics and home-appliance store," Public e-commerce datasets. [Online]. Available: https://data.rees46.com/. [Accessed: Aug. 3, 2026].

[5] Amazon Web Services, "Using managed scaling in Amazon EMR," Amazon EMR Management Guide. [Online]. Available: https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-managed-scaling.html. [Accessed: Aug. 3, 2026].

[6] Apache Software Foundation, "Structured Streaming programming guide, Spark 3.5.6." [Online]. Available: https://spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html. [Accessed: Aug. 3, 2026].

SUBMISSION LINKS

GitHub repository:
https://github.com/Sharmila-Ramaraj/scalable-clickstream-analytics

Demo video: to be added before assessment submission

Live AWS dashboard: http://scalable-real-time-clickstream-analytics-x24244066.s3-website-us-east-1.amazonaws.com